

DỰ ÁN HỌC MÁY TỪ ĐẦU ĐẾN CUỐI

Chương 3 - Phân loại (Classification)

TS. Trần Thành Thắng

Đại học Đông Á

August 11, 2026

Nội dung Chương trình

1. Giới thiệu Phân loại, MNIST & Đánh giá Hiệu suất cơ bản
2. Sự đánh đổi Hiệu suất & Phân loại Đa lớp
3. Phân tích lỗi, Đa nhãn & Đa đầu ra

- Nắm vững khái niệm Phân loại (Classification) trong Học máy.
- Hiểu rõ và thực hành với bộ dữ liệu MNIST ("Hello World" của học máy).
- Đánh giá mô hình bằng các thước đo hiệu suất (Ma trận nhầm lẫn, Precision, Recall, F1-score, ROC AUC).
- Phân biệt và thực hiện Phân loại Đa lớp (Multiclass), Đa nhãn (Multilabel) và Đa đầu ra (Multioutput).
- Kỹ năng phân tích lỗi để cải thiện hiệu suất mô hình.

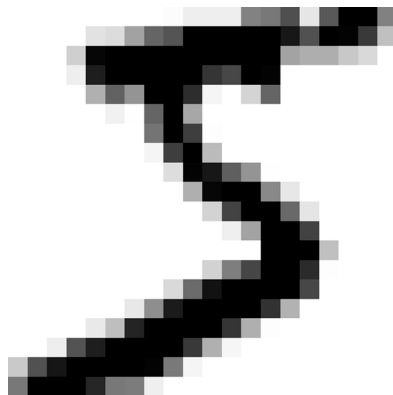
1. Giới thiệu Phân loại, MNIST & Đánh giá Hiệu suất cơ bản

Giới thiệu Bộ dữ liệu MNIST

- Các tác vụ học có giám sát phổ biến nhất là Hồi quy (Regression) và Phân loại (Classification).
- Chúng ta sẽ sử dụng bộ dữ liệu **MNIST**, gồm 70.000 hình ảnh nhỏ của các chữ số viết tay bởi học sinh và nhân viên Cục Thống kê Hoa Kỳ.
- MNIST được nghiên cứu nhiều đến mức nó được gọi là **"Hello World" của học máy**.
- Mục tiêu: Phân loại (nhận dạng) chữ số trong từng bức ảnh.

Hình ảnh chữ số MNIST

- Mỗi hình ảnh có kích thước 28x28 pixel, tương đương 784 đặc trưng (pixels).
- Mỗi pixel đại diện cho cường độ xám, từ 0 (trắng) đến 255 (đen).
- Trong ảnh là một ví dụ trực quan hóa một chữ số 5 từ bộ dữ liệu.



Hình 3-1. Ví dụ về hình ảnh MNIST

Sự đa dạng của chữ số MNIST

- Các chữ số được viết với nhiều phong cách, nét thanh, nét đậm khác nhau.
- Để minh họa độ phức tạp của bài toán phân loại, hình bên cạnh liệt kê một số chữ số mẫu từ tập dữ liệu.



Hình 3-2. Các chữ số từ bộ dữ liệu MNIST

Phân chia tập huấn luyện và tập kiểm thử

- **Nguyên tắc vàng:** Luôn tạo một tập kiểm thử (Test set) và tách biệt nó trước khi kiểm tra kỹ lưỡng dữ liệu.
- May mắn là tập dữ liệu MNIST tải từ OpenML đã được chia sẵn:
 - Tập huấn luyện (Train set): 60.000 hình ảnh đầu tiên.
 - Tập kiểm thử (Test set): 10.000 hình ảnh cuối cùng.
- Tập huấn luyện đã được xáo trộn (shuffled), đảm bảo các thuật toán học (đặc biệt là các thuật toán học tuần tự) không bị nhận nhiều trường hợp giống nhau liên tiếp.

Huấn luyện bộ phân loại nhị phân (Binary Classification)

Bài toán: Bộ phát hiện chữ số 5

- Hãy đơn giản hóa bài toán: Chúng ta chỉ cần phát hiện xem một chữ số có phải là "5" hay không.
- Đây là ví dụ về **Phân loại Nhị phân (Binary Classification)**: Phân biệt giữa 2 lớp (5 và không-phải-5).
- Mục tiêu (Label) mới: True (nếu là số 5), False (nếu là số khác).

Thuật toán Stochastic Gradient Descent (SGD)

- SGD (Xuống dốc ngẫu nhiên) là một thuật toán tối ưu rất hiệu quả để xử lý các tập dữ liệu lớn.
- SGD xử lý các trường hợp huấn luyện độc lập, từng trường hợp một (phù hợp với học trực tuyến - online learning).
- Lớp `SGDClassifier` trong Scikit-Learn giúp ta dễ dàng huấn luyện mô hình này.

Các thước đo hiệu suất

Đo độ chính xác bằng kiểm định chéo

- Giống như hồi quy, ta có thể dùng kiểm định chéo (Cross-Validation) để đánh giá mô hình phân loại.
- **Kết quả:** SGDClassifier đạt độ chính xác (Accuracy) trên 95% cho mọi fold!
- Nghe có vẻ tuyệt vời, nhưng hãy xem xét cẩn thận hơn.

Sự đánh lừa của Độ chính xác (Accuracy)

- Nếu ta tạo một "bộ phân loại ngốc nghếch" (Dummy Classifier) luôn đoán "không-phải-5", thì độ chính xác của nó cũng trên 90%!
- Tại sao? Vì chỉ có 10% chữ số trong tập dữ liệu là số 5.
- **Kết luận:** Độ chính xác (Accuracy) không phải là thước đo tốt cho các **tập dữ liệu bị lệch (skewed datasets)**.

Ma trận nhầm lẫn (Confusion Matrix)

- Ma trận nhầm lẫn (CM) là một cách tốt hơn nhiều để đánh giá hiệu suất.
- Ý tưởng chung: Đếm số lần các trường hợp của lớp A bị phân loại nhầm thành lớp B.
- Mỗi hàng là một lớp thực tế, mỗi cột là một lớp được dự đoán.

Cấu trúc của Ma trận nhầm lẫn

- **True Negative (TN):** Âm tính thật.
- **False Positive (FP):** Dương tính giả (Lỗi loại I).
- **False Negative (FN):** Âm tính giả (Lỗi loại II).
- **True Positive (TP):** Dương tính thật.

		Predicted		
		Negative	Positive	
Actual	Negative	8 3 9	6	Precision (e.g., 3 out of 4)
	Positive	7 2	5 5 5	
		Recall (e.g., 3 out of 5)		
		TN	FP	
		FN	TP	

Hình 3-3. Ma trận nhầm lẫn minh họa

Độ chính xác (Precision) và Độ nhạy (Recall)

- **Precision (Độ chính xác của lớp Dương tính):** Trong số các trường hợp mô hình dự đoán là Tích cực (5), có bao nhiêu phần trăm là đúng?

$$Precision = \frac{TP}{TP + FP}$$

- **Recall (Độ nhạy / Tỷ lệ dương tính đúng):** Trong số các trường hợp Tích cực thực sự, mô hình đã phát hiện được bao nhiêu phần trăm?

$$Recall = \frac{TP}{TP + FN}$$

Sự kết hợp: Điểm F1 (F1 Score)

- Điểm F1 là **trung bình điều hòa (Harmonic Mean)** của Precision và Recall.
- Điểm F1 chỉ cao khi CẢ Precision và Recall đều cao.

$$F1 = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}} = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- Điểm F1 ưu tiên các bộ phân loại có Precision và Recall tương tự nhau.

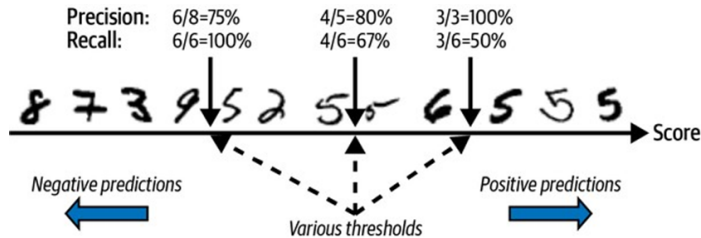
2. Sự đánh đổi Độ chính xác và Độ nhạy

Ngưỡng quyết định (Decision Threshold)

- Làm sao mô hình quyết định một mẫu là Tích cực hay Tiêu cực?
- Nó tính toán một **điểm số quyết định (decision score)**.
- Nếu điểm số $>$ Ngưỡng (Threshold) $\rightarrow$ Tích cực.
- Nếu điểm số $<$ Ngưỡng (Threshold) $\rightarrow$ Tiêu cực.

Minh họa sự đánh đổi (Trade-off)

- Nếu **Tăng ngưỡng**: Giảm False Positive (Precision tăng), nhưng tăng False Negative (Recall giảm).
- Nếu **Giảm ngưỡng**: Giảm False Negative (Recall tăng), nhưng tăng False Positive (Precision giảm).

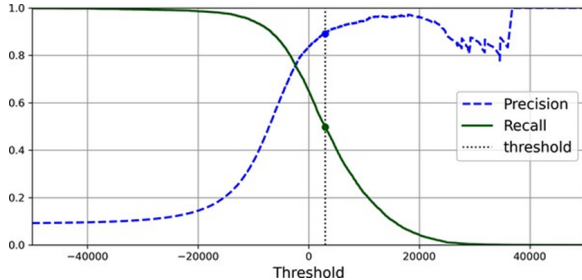


Hình 3-4. Sự đánh đổi độ chính xác/độ nhạy theo ngưỡng quyết định

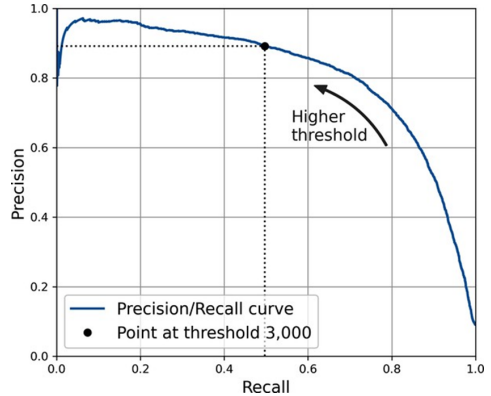
Các biểu đồ phân tích Precision/Recall

- Để chọn được một Threshold phù hợp nhất với bài toán thực tế, ta cần trực quan hóa Precision và Recall.
- **Đường cong số 1:** Vẽ Precision và Recall theo từng giá trị ngưỡng (Threshold).
- **Đường cong số 2:** Vẽ trực tiếp Precision theo Recall để xem đường tiệm cận.
- Ta nên chọn điểm ngưỡng ngay trước khi Precision bị sụt giảm quá mạnh (ví dụ: ở Recall khoảng 60%-80%).

Minh họa: Đường cong Precision/Recall



Hình 3-5. Precision và Recall so với Threshold



Hình 3-6. Precision so với Recall

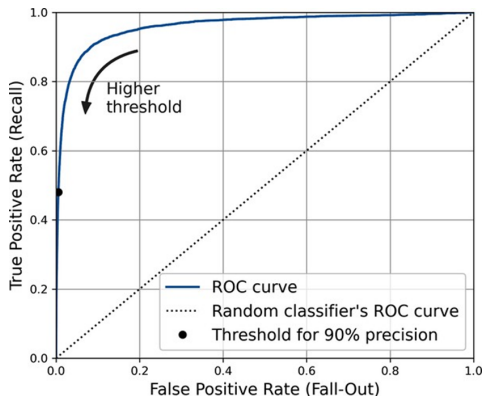
Đường cong ROC (Receiver Operating Characteristic)

- Rất phổ biến trong bộ phân loại nhị phân.
- Vẽ biểu đồ **Tỷ lệ dương tính đúng (TPR / Recall)** so với **Tỷ lệ dương tính giả (FPR / Fall-out)**.
- FPR là tỷ lệ các mẫu Âm tính bị nhận diện sai là Dương tính.
- Sự đánh đổi: TPR càng cao (mô hình càng nhạy), thì FPR càng cao (càng nhiều báo động giả).

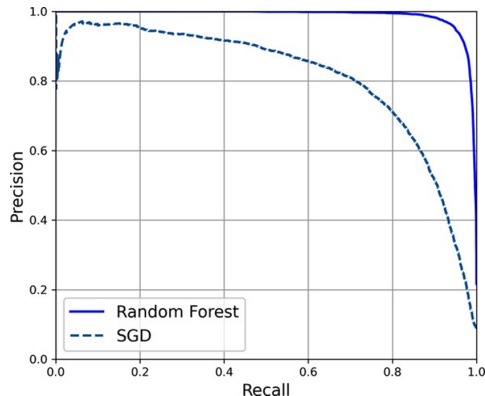
ROC AUC: So sánh các bộ phân loại

- Để so sánh 2 mô hình, ta đo **Diện tích dưới đường cong (AUC - Area Under the Curve)**.
- Bộ phân loại hoàn hảo: $\text{ROC AUC} = 1$.
- Bộ phân loại ngẫu nhiên: $\text{ROC AUC} = 0.5$.
- Mô hình có ROC gần góc trên bên trái nhất (AUC cao) là mô hình tốt nhất.

Minh họa: Đường cong ROC của SGD và Random Forest



Hình 3-7. Đường cong ROC của mô hình SGD



Hình 3-8. So sánh ROC: SGD vs Random Forest

Phân loại Đa lớp (Multiclass Classification)

Chiến lược Một-đối-phần-còn-lại (OvR / OvA)

- Phân biệt giữa nhiều hơn hai lớp.
- **OvR (One-vs-Rest)**: Huấn luyện 10 bộ phân loại nhị phân (từ chữ số 0 đến 9).
- Bộ phân loại "0": Số 0 vs Các số còn lại.
- Khi phân loại: Lấy điểm quyết định từ cả 10 mô hình và chọn lớp có điểm cao nhất.

Chiến lược Một-đối-một (OvO)

- **OvO (One-vs-One):** Huấn luyện một bộ phân loại nhị phân cho mỗi CẶP chữ số.
- 0 vs 1, 0 vs 2, 1 vs 2...
- Tổng số mô hình cần huấn luyện: $N \times (N - 1) / 2 = 45$ bộ phân loại!
- Phù hợp cho các thuật toán khó mở rộng với lượng dữ liệu lớn (như Support Vector Machine - SVC).

Phân loại đa lớp tự động trong Scikit-Learn

- Scikit-Learn tự động phát hiện khi ta dùng mô hình nhị phân cho dữ liệu đa lớp.
- Nó tự động chọn OvO hoặc OvR tùy thuộc vào thuật toán.
- Ví dụ: Truyền toàn bộ các nhãn y_{train} (từ 0-9) vào mô hình Support Vector Machine (SVC), Scikit-Learn sẽ tự động huấn luyện bằng phương pháp OvO.

Phân loại đa lớp với thuật toán gốc

- Một số mô hình sinh ra đã hỗ trợ Phân loại đa lớp (ví dụ: `RandomForestClassifier`, `SGDClassifier`, `GaussianNB`).
- Trong trường hợp này, Scikit-Learn không cần phải tạo ra các thuật toán nhị phân giả lập (OvO/OvR).

Điểm số quyết định cho dự đoán đa lớp

- Khi chạy `decision_function()` cho đa lớp, mô hình sẽ trả về 1 mảng điểm tương ứng với mọi lớp có thể.
- Điểm cao nhất nằm ở vị trí (index) nào thì mô hình sẽ dự đoán ảnh thuộc về lớp đó.
- Danh sách các lớp mục tiêu được lưu trong thuộc tính `classes_`.

Đánh giá bộ phân loại đa lớp

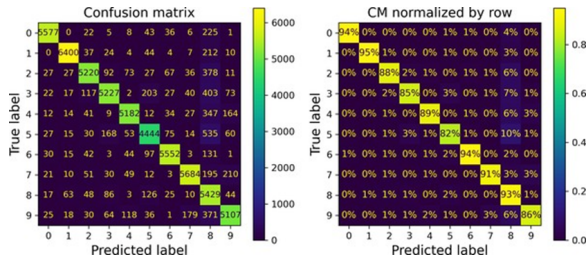
- Ta vẫn có thể sử dụng hàm `cross_val_score` để đánh giá độ chính xác như phân loại nhị phân.
- Tuy nhiên, để cải thiện kết quả, cần kết hợp với các bước **Chuẩn bị Dữ liệu** ở Chương 2.
- Ví dụ: Chuẩn hóa tỉ lệ các đặc trưng (Feature Scaling) bằng `StandardScaler`.

3. Phân tích Lỗi & Các bài toán nâng cao

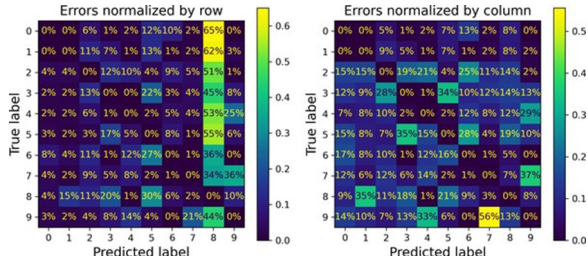
Trực quan hóa Ma trận nhầm lẫn đa lớp

- Đối với bài toán phân loại đa lớp (10 lớp), ma trận nhầm lẫn sẽ có kích thước 10×10 . Rất khó để nhìn bằng mắt thường.
- Cách tốt nhất là trực quan hóa nó bằng biểu đồ màu (heat map).
- Ở biểu đồ chuẩn, phần lớn dữ liệu hội tụ trên đường chéo (nghĩa là dự đoán đúng).
- Để xem phần LỖI rõ hơn, ta tập trung trọng số vào các ô "dự đoán sai".

Minh họa: Phân tích Ma trận nhầm lẫn



Hình 3-9. Ma trận nhầm lẫn được chuẩn hóa



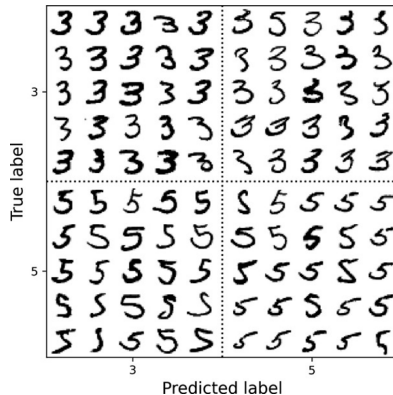
Hình 3-10. Ma trận nhầm lẫn chỉ hiển thị Lỗi

Phân tích lỗi riêng lẻ và cách khắc phục

- Nhìn vào biểu đồ lỗi, ta có thể thấy số 8 thường bị nhầm lẫn nhiều nhất.
- **Cách khắc phục:**
 - Thu thập thêm dữ liệu số 8 để mô hình học rõ hơn.
 - Viết thuật toán tạo đặc trưng (ví dụ: đếm số vòng kín của số 8).
 - Tiền xử lý hình ảnh, tăng độ tương phản...

So sánh lỗi giữa số 3 và 5

- SGDClassifier là một mô hình tuyến tính, nó chỉ tổng hợp điểm số dựa trên trọng số pixel.
- Hai số 3 và 5 rất giống nhau (chỉ lệch vài pixel nổi cung).
- Hình bên cạnh cho thấy các lỗi có thể do chữ viết tay quá xấu, dẫn đến mô hình bối rối.



Hình 3-11. Sự nhầm lẫn giữa chữ số 3 và 5

Tăng cường dữ liệu (Data Augmentation)

- Mô hình máy học rất nhạy cảm với việc hình ảnh bị lệch (shift) hoặc xoay (rotate).
- Giải pháp hiệu quả: **Tăng cường dữ liệu (Data Augmentation)**.
- Biến đổi nhẹ các bức ảnh huấn luyện (dịch trái, dịch phải 1 pixel...) rồi thêm vào tập huấn luyện.
- Giúp mô hình có khả năng chịu đựng sai số (robust) tốt hơn!

Phân loại Đa nhãn (Multilabel Classification)

Phân loại Đa nhãn (Multilabel) là gì?

- Một mẫu (instance) duy nhất nhưng thuộc về **nhều lớp** cùng lúc.
- Ví dụ: Nhận diện khuôn mặt. Một bức ảnh có chứa cả Alice và Charlie, nhưng không có Bob.
- Đầu ra của mô hình sẽ là một chuỗi cờ Nhị phân (Binary Tags), ví dụ: [True, False, True].

Ví dụ về hệ thống đa nhãn

- Áp dụng vào MNIST: Xây dựng bộ phân loại đánh giá 2 tiêu chí cùng lúc:
 - Chữ số đó có lớn không? (7, 8, 9) $\rightarrow$ `y_large`
 - Chữ số đó có phải là số lẻ không? (1, 3, 5, 7, 9) $\rightarrow$ `y_odd`
- Kết hợp 2 nhãn này lại: `y_multilabel`. Mô hình `KNeighborsClassifier` hỗ trợ xử lý việc này một cách tự nhiên.

Đánh giá hệ thống phân loại đa nhãn

- Tính **F1-score** cho từng nhãn (từng tiêu chí) riêng biệt, sau đó lấy trung bình của tất cả các nhãn.
- Tùy chọn `average="macro"` giả định mọi nhãn quan trọng như nhau.
- Tùy chọn `average="weighted"` sẽ đánh trọng số dựa trên số lượng mẫu thực tế của từng nhãn (nếu tập dữ liệu bị mất cân bằng).

Phân loại đa nhãn với ClassifierChain

- Nếu dùng mô hình như SVC (không hỗ trợ multilabel tự nhiên), ta có thể huấn luyện 1 mô hình cho mỗi nhãn.
- Tuy nhiên, các nhãn có thể phụ thuộc nhau (Ví dụ: số lớn > 7 thì có tỷ lệ số chẵn cao hơn).
- Lớp ClassifierChain giải quyết bằng cách tổ chức mô hình thành một **chuỗi (chain)**: Mô hình A dự đoán xong sẽ đưa kết quả làm đầu vào phụ cho Mô hình B dự đoán tiếp.

Phân loại Đa đầu ra (Multioutput Classification)

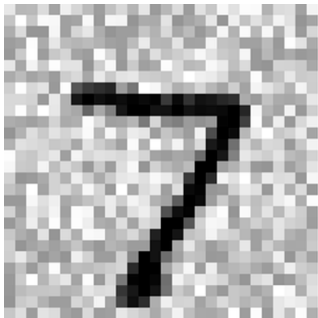
Phân loại Đa đầu ra là gì?

- Khái quát hóa của Phân loại đa nhãn: Đầu ra chứa nhiều nhãn, nhưng mỗi nhãn có thể là **đa lớp** (nhận nhiều giá trị khác nhau).
- Ứng dụng: **Khử nhiễu hình ảnh (Noise removal)**.
- Đầu vào: 1 bức ảnh MNIST bị làm nhiễu pixel.
- Đầu ra: 784 pixels, mỗi pixel (là 1 nhãn) nhận một giá trị từ 0 đến 255 (Đa lớp).

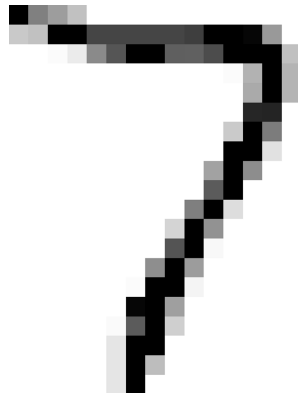
Bài toán khử nhiễu hình ảnh

- Ta cố tình thêm nhiễu (noise) ngẫu nhiên vào dữ liệu huấn luyện. Mục tiêu mong muốn chính là hình ảnh MNIST ban đầu chưa nhiễu.
- Mô hình `KNeighborsClassifier` sẽ học cách so khớp bức ảnh nhiễu và tìm lại bức ảnh gần gốc nhất.
- Xem minh họa quá trình thực hiện ở Slide tiếp theo.

Minh họa: Quá trình khử nhiễu



Hình 3-12. Ảnh bị nhiễu và Ảnh mục tiêu



Hình 3-13. Ảnh đã được mô hình làm sạch

- Hiểu rõ các chỉ số đánh giá cho Bài toán Phân loại (Precision, Recall, F1, ROC).
- Kiểm soát và điều chỉnh được sự đánh đổi Precision/Recall bằng Threshold.
- Nắm được quy trình Phân tích lỗi (Error Analysis) qua Ma trận nhầm lẫn.
- Áp dụng các kỹ thuật Phân loại Đa lớp (OvR/OvO), Đa nhãn và Đa đầu ra cho các bài toán phức tạp.